บทที่ 7

การตกแต่งอินเทอร์เฟซ ไอคอน และรูปภาพประสิทธิภาพสูง

การออกแบบประสบการณ์ผู้ใช้งาน (User Experience - UX) ที่ดีไม่ได้ขึ้นอยู่กับฟังก์ชันการทำงานที่ครบถ้วนเพียงอย่างเดียว แต่ยังขึ้นอยู่กับความลื่นไหล ความเร็วในการจัดแสดงรูปภาพ และความกลมกลืนของการออกแบบดีไซน์ ในบทเรียนนี้เราจะศึกษาเชิงลึกเกี่ยวกับการควบคุมสไตล์ขั้นสูงด้วยระบบจัดตำแหน่ง **absolute**, การโหลดแบบอักษรพิเศษ (Custom Fonts), การเรียกใช้งานชุดเวกเตอร์ไอคอน และการเพิ่มประสิทธิภาพการแสดงภาพด้วย **expo-image** รวมถึงระบบปรับสีตามธีมของตัวเครื่อง (Dark Mode)

7.1 ระบบพิกัดตำแหน่งสัมพัทธ์และสัมบูรณ์ (Relative and Absolute Positioning)

ในการจัดองค์ประกอบหน้าจอบน React Native ค่าเริ่มต้นของสไตล์ **position** จะเป็น **'relative'** (พิกัดตำแหน่งสัมพัทธ์) ซึ่งสอดคล้องกับการวางตัวตนต่อท้ายกันตามทิศทางของ Flexbox (React Native, 2024) แต่สำหรับการออกแบบ UI ยุคใหม่ เช่น การทำเครื่องหมายแจ้งเตือน (Notification Badge) ทับบนไอคอน หรือการวางปุ่มลอย (Floating Action Button) ค้างไว้ที่มุมขวาล่าง เราจำเป็นต้องเลือกใช้ **'absolute'** (พิกัดตำแหน่งสัมบูรณ์)

ข้อแตกต่างและการประยุกต์ใช้งาน

• **`relative` (Default):** คอมโพเนนต์จะจัดวางพื้นที่และขยับตำแหน่งโดยอ้างอิงจากตำแหน่งที่มันควรจะอยู่ปกติในระบบ Flow โดยที่ยังคงจองพื้นที่ของตัวเองไว้ ไม่ให้ชิ้นงานอื่นมาเบียดทับ

• **`absolute`:** คอมโพเนนต์จะหลุดออกจากระเบียบการจัดแถวของ Flexbox ทันที โดยพิกัดจะถูกระบุด้วยทิศทาง **top**, **bottom**, **left**, **right** สัมพันธ์กับพื้นที่ของ **"คอนเทนเนอร์แม่ตัวแรกสุดที่มีระดับชั้นอยู่เหนือมันและทำตัวเป็นจุดอ้างอิง"** (มักจะตั้งค่าแม่เป็น **position: 'relative'** หรือปล่อยไว้เป็นดีฟอลต์)

```
import { StyleSheet, View, Text } from 'react-native';
import { Ionicons } from '@expo/vector-icons';

export default function BadgeExample() {
  return (
    <View style={styles.container}>
      {/* 1. ปุ่มกระดิ่งแจ้งเตือน */}
      <View style={styles.iconButton}>
        <Ionicons name="notifications" size={32} color="#ffffff" />
```

```
        {/* 2. ตัวเลขสีแดงแจ้งเตือนวางทับซ้อนมุมขวาบน */}
        <View style={styles.badge}>
          <Text style={styles.badgeText}>3</Text>
        </View>
    </View>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    backgroundColor: '#0f172a',
  },
  iconButton: {
    width: 60,
    height: 60,
    borderRadius: 30,
    backgroundColor: '#1e293b',
    justifyContent: 'center',
    alignItems: 'center',
    position: 'relative', // เป็นหลักแหล่งพิกัดอ้างอิงให้ลูกสะกดตำแหน่ง
    borderWidth: 1,
    borderColor: '#334155',
  },
  badge: {
    position: 'absolute', // หลุดออกจากโฟลว์เพื่อทับซ้อนอิสระ
    top: -4,              // ดันขึ้นไปด้านบน 4px เหนือขอบกระดิ่ง
    right: -4,            // ขยับเยื้องขวา 4px เลยขอบกระดิ่ง
    backgroundColor: '#ef4444', // สีแดงเตือนภัย
    minWidth: 20,
    height: 20,
    borderRadius: 10,
    justifyContent: 'center',
    alignItems: 'center',
    paddingHorizontal: 4,
  },
  badgeText: {
    color: '#ffffff',
    fontSize: 12,
    fontWeight: 'bold',
  },
});
```

7.2 การใช้งาน Custom Fonts ด้วย expo-font

ฟอนต์ดั้งเดิมของระบบปฏิบัติการ (System Fonts) มักจะขาดเอกลักษณ์เฉพาะตัวของแบรนด์ โครงการ Expo SDK 54 สนับสนุนการดาวน์โหลดชุดฟอนต์จากภายนอกมาติดตั้งใช้งานในระดับเนทีฟอย่างสะดวก ผ่านไลบรารี **expo-font** (Expo, 2024) และจัดการความสมบูรณ์ในการโหลดฟอนต์พร้อมกับระบบ Splash Screen เพื่อป้องกันปัญหาสัญลักษณ์เบี้ยวหรือกระตุก (Flash of Unstyled Text)

การดาวน์โหลดและติดตั้งคอมโพเนนต์

ขั้นตอนแรกในการทำงานคือการติดตั้งไลบรารีที่จำเป็น:

```
npx expo install expo-font expo-splash-screen
```

การเขียนโค้ดผูกฟอนต์และหน่วงเวลา Splash Screen

เราต้องสั่งหยุดไม่ให้ Splash Screen ปิดตัวเองก่อนที่ฟอนต์จะถูกโหลดเสร็จลงในหน่วยความจำ RAM ของเครื่อง โดยเรียกใช้ `SplashScreen.preventAutoHideAsync()` และสั่งซ่อนด้วย `SplashScreen.hideAsync()` หลังจากผลลัพธ์การโหลดพร้อมรัน:

```
import { useEffect } from 'react';
import { View, Text, StyleSheet } from 'react-native';
import { useFonts } from 'expo-font';
import * as SplashScreen from 'expo-splash-screen';

// 1. ตรึงภาพ Splash Screen ไว้ไม่ให้หน้าจอดับลงก่อนการโหลด
SplashScreen.preventAutoHideAsync();

export default function FontLoaderApp() {
 // 2. เรียกใช้ useFonts โหลดแบบอักษรพิเศษจากโฟลเดอร์ assets
  const [fontsLoaded, error] = useFonts({
    'Prompt-Regular': require('../../assets/fonts/Prompt-Regular.ttf'),
    'Prompt-Bold': require('../../assets/fonts/Prompt-Bold.ttf'),
  });

  useEffect(() => {
  // 3. เมื่อดาวน์โหลดเสร็จสมบูรณ์หรือมีข้อผิดพลาด ให้ทำการซ่อน Splash Screen
    if (fontsLoaded || error) {
      SplashScreen.hideAsync();
    }
  }, [fontsLoaded, error]);

 // 4. หากฟอนต์ยังไม่พร้อมและไม่พบข้อผิดพลาด ให้คืนค่าว่างเปล่า (null) เพื่อป้องกันการเรนเดอร์พัง
  if (!fontsLoaded && !error) {
    return null;
  }
```

```
  return (
    <View style={styles.container}>
      <Text style={styles.textRegular}>สวัสดีภาษาไทยด้วยแบบอักษร Prompt
(Regular)</Text>
      <Text style={styles.textBold}>หัวข้อสำคัญด้วย Prompt (Bold)</Text>
    </View>
 );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    padding: 24,
    backgroundColor: '#0f172a',
  },
  textRegular: {
    fontFamily: 'Prompt-Regular', // อ้างอิงตรงตามชื่อคีย์ที่ลงทะเบียนใน useFonts
    fontSize: 18,
    color: '#cbd5e1',
    marginBottom: 12,
  },
  textBold: {
    fontFamily: 'Prompt-Bold',
    fontSize: 24,
    color: '#ffffff',
  },
});
```

7.3 การใช้งานระบบเวกเตอร์ไอคอนด้วย @expo/vector-icons

ไลบรารี **@expo/vector-icons** (Expo, 2024) ติดตั้งมาพร้อมใช้งานในแม่แบบโครงการของ Expo ช่วยลดภาระการโหลดไฟล์ PNG ของไอคอน ซึ่งทำให้ขนาดไฟล์แอปพลิเคชันใหญ่โดยไม่จำเป็น โดยชุดแพ็คเกจนี้จะรวมไอคอนเวกเตอร์ชื่อดังไว้เพียบ เช่น Ionicons, AntDesign, FontAwesome, MaterialIcons, Feather และ Octicons

เครื่องมือสำหรับค้นหาไอคอนอย่างเป็นทางการ

ผู้พัฒนาสามารถเลือกค้นหาชุดภาพและรายชื่อไอคอนที่ถูกต้องได้ผ่านเว็บไซต์ icons.expo.fyi ซึ่งจะบอกคำสั่ง Import และรายชื่อไอคอนทั้งหมด

ตัวอย่างการประยุกต์ใช้งาน

```
import { View, StyleSheet, TouchableOpacity, Text } from 'react-native';
```

```
import { Ionicons, Feather } from '@expo/vector-icons';

export default function MenuGrid() {
  return (
    <View style={styles.grid}>
      {/* การใช้งาน Ionicons สำหรับปุ่มตั้งค่า */}
      <TouchableOpacity style={styles.card}>
        <Ionicons name="settings-outline" size={28} color="#38bdf8" />
        <Text style={styles.label}>ตั้งค่าระบบ</Text>
      </TouchableOpacity>

      {/* การใช้งาน Feather สำหรับปุ่มแผนที่ */}
    <TouchableOpacity style={styles.card}>
        <Feather name="map-pin" size={28} color="#f43f5e" />
        <Text style={styles.label}>สถานที่</Text>
      </TouchableOpacity>
    </View>
 );
}

const styles = StyleSheet.create({
  grid: {
    flexDirection: 'row',
    justifyContent: 'space-around',
    padding: 20,
    backgroundColor: '#0f172a',
  },
  card: {
    backgroundColor: '#1e293b',
    padding: 16,
    borderRadius: 12,
    alignItems: 'center',
    width: '40%',
    borderWidth: 1,
    borderColor: '#334155',
  },
  label: {
    color: '#ffffff',
    marginTop: 8,
    fontSize: 14,
  },
});
```

7.4 การจัดการรูปภาพประสิทธิภาพสูงด้วย expo-image

คอมโพเนนต์ `Image` ดั้งเดิมของ React Native ประสบปัญหาร้ายแรงหลายประการเมื่อต้องทำงานกับภาพจากเน็ตเวิร์กจำนวนมาก เช่น การกระพริบตา (Flickering) เวลาอัปเดตข้อมูล, ไม่มีระบบเซฟเก็บความจำสำรองบนความจุฮาร์ดดิสก์แบบเนทีฟ (Native Caching) ทำให้แอปเปลืองอินเทอร์เน็ต และการเปิดโหลดภาพขนาดใหญ่ก่อให้เกิดอาการค้างชั่วขณะบนโทรศัพท์ Android ระดับล่าง

เพื่อขจัดขีดจำกัดดังกล่าว Expo จึงพัฒนาคอมโพเนนต์ **`expo-image`** (Expo, 2024) ขึ้นมาโดยขับเคลื่อนผ่านไลบรารีประสิทธิภาพสูงของภาษาเนทีฟอย่าง **SDWebImage (iOS)** และ **Glide (Android)**

ความสำคัญของ expo-image

• **ระบบ Caching ชั้นเลิศ:** บันทึกภาพลงบนไดรฟ์เครื่องอัตโนมัติ เปิดซ้ำไม่ต้องโหลดเน็ตอีกครั้ง

• **ความเร็วในการประมวลผลสูง:** ป้องกันไม่ให้แอปพลิเคชันกระตุกขณะผู้ใช้ทำการปัดเลื่อนลิสต์ที่มีภาพประกอบ

• **สอดคล้องกับมาตรฐานเว็บ:** การใช้งานฟังก์ชันขยายสไตล์สอดรับกับคุณสมบัติของ CSS `object-fit` (เช่นการใช้ค่า `contentFit`)

• **รองรับหลากหลายไฟล์สมัยใหม่:** สามารถรันภาพเคลื่อนไหวอย่าง WebP, APNG, GIF รวมถึงไฟล์ความหนาแน่นสูงอย่าง AVIF

```
npx expo install expo-image
```

7.5 การใช้งาน Blurhash Placeholder และ Transition

สองกลยุทธ์สำคัญที่จะช่วยเปลี่ยนแอปที่ดูธรรมดาให้เป็นแอปเกรดพรีเมียม คือการจัดการรอยต่อของเวลาที่รูปภาพกำลังโหลด:

1. ระบบจำลองภาพแบบเลือนลาง (Blurhash Placeholder)

Blurhash คือเทคโนโลยีการแปลงภาพถ่ายขนาดใหญ่ให้กลายเป็นรหัสข้อความสั้น ๆ (เช่น `L6PZUpUR00-y_4V[00aeY5g2o1of`) ซึ่งสามารถวาดแสดงผลขึ้นมาเป็นภาพไล่เฉดสีเบลอ ๆ ได้ในเศษเสี้ยววินาทีโดยไม่ต้องรอเน็ตเวิร์ก ช่วยลดความอึดอัดของผู้ใช้เมื่อเปรียบเทียบกับวงล้อโหลดข้อมูลที่ว่างเปล่า

2. เอฟเฟกต์ Fade-in Transition

เมื่อระบบดาวน์โหลดภาพไฟล์จริงจากเซิร์ฟเวอร์เสร็จสิ้นแล้ว expo-image จะทำการเปลี่ยนผ่านจากภาพเบลอตัวพรีวิวไปสู่รูปจริงอย่างราบรื่นด้วยเอฟเฟกต์อนิเมชันแบบสลัวทีละนิด ช่วยลดความตระหนกสายตาของผู้ใช้ (No Flickering Jump)

```
import { View, StyleSheet, Text } from 'react-native';
import { Image } from 'expo-image';

// ตัวอย่างสตริง Blurhash จำลองเฉดสีเพื่อรอโหลด
const sampleBlurhash = 'L6PZUpUR00-y_4V[00aeY5g2o1of';

export default function HighPerformanceImage() {
  return (
    <View style={styles.container}>
      <Text style={styles.header}>ตัวอย่างการแสดงผลรูปภาพผ่าน expo-image</Text>

      <Image
        style={styles.image}
        source="https://picsum.photos/id/237/800/600"
        placeholder={{ blurhash: sampleBlurhash }} // แสดงภาพพรีวิวเบลอตามรหัสชุดข้อความทันที
        contentFit="cover" // เทียบเท่า object-fit: cover ในระบบ CSS
        transition={600}    // เล่นเอฟเฟกต์เฟดภาพจริง 0.6 วินาทีเพื่อความนุ่มนวล
      />
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    backgroundColor: '#0f172a',
  },
  header: {
    color: '#94a3b8',
    marginBottom: 16,
    fontSize: 15,
  },
  image: {
    width: 300,
    height: 200,
    borderRadius: 16,
  },
});
```

7.6 การรองรับโหมดมืด (Dark Mode / Theme Support) ด้วย useColorScheme

ในโทรศัพท์มือถือยุคปัจจุบัน ระบบปฏิบัติการจะเปิดโอกาสให้ผู้ใช้อัปเกรดหน้าจอไปใช้โทนสีดำ (Dark Mode) เพื่อความปลอดภัยของสายตาและประหยัดพลังงาน แบตเตอรี่ สำหรับแอป React Native เราสามารถเขียนตรวจสอบสถานะสีเครื่องได้โดยตรงผ่าน `useColorScheme` Hook ของ `react-native`

การประกาศตัวแปรสีและผูกใช้ในสไตล์

เมื่อตรวจพบการปรับระบบ โค้ดหน้าจอจะเข้าข่ายและแสดงธีมที่สลับค่าโทนสีอย่างไดนามิก:

```
import { StyleSheet, Text, View, useColorScheme } from 'react-native';

export default function DynamicThemeScreen() {
 // 1. เรียกรับค่าสถานะโหมดระบบปฏิบัติการ ('light' หรือ 'dark')
  const colorScheme = useColorScheme();

 // 2. ตรวจเช็คเพื่อกำหนดธีมสีแบบ Dynamic
  const isDarkMode = colorScheme === 'dark';

  return (
  // 3. กำหนดสไตล์โดยสลับ Object สไตล์ตามสถานะ
    <View style={[styles.container, isDarkMode ? styles.darkContainer :
styles.lightContainer]}>
      <Text style={[styles.title, isDarkMode ? styles.darkText :
styles.lightText]}>
        {isDarkMode ? '🌙 ระบบของคุณเปิดโหมดมืดอยู่' : '🌞 ระบบของคุณเปิดโหมดสว่างอยู่'}
      </Text>
      <Text style={[styles.subtitle, isDarkMode ? styles.darkSub :
styles.lightSub]}>
    สไตล์และโทนสีของแอปตัวนี้จะขยับสับเปลี่ยนตามค่าหน้าจอหลักของสมาร์ทโฟน
      </Text>
    </View>
 );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    padding: 24,
  },
 // ธีมสีพื้นหลังสว่าง/มืด
```

```
  lightContainer: {
    backgroundColor: '#f8fafc',
  },
  darkContainer: {
    backgroundColor: '#0f172a',
  },
  title: {
    fontSize: 22,
    fontWeight: 'bold',
    marginBottom: 10,
    textAlign: 'center',
  },
  subtitle: {
    fontSize: 15,
    textAlign: 'center',
    lineHeight: 22,
  },
 // ธีมตัวหนังสือ
  lightText: {
    color: '#0f172a',
  },
  darkText: {
    color: '#ffffff',
  },
  lightSub: {
    color: '#64748b',
  },
  darkSub: {
    color: '#94a3b8',
  },
});
```

สรุปท้ายบทเรียน

ระบบพิกัดตำแหน่งสัมพัทธ์และสัมบูรณ์: การจัดวางอินเทอร์เฟซใน React Native ใช้สไตล์ position ในการกำหนดระบบพิกัด โดยมี relative เป็นค่าเริ่มต้นที่จัดเรียงวัตถุตามระบบ Flexbox โดยยังจองพื้นที่ไว้ และ absolute สำหรับยกวัตถุออกจากโฟลว์เพื่อกำหนดทิศทางผ่าน top, bottom, left และ right สัมพันธ์กับพื้นที่ของคอมโพเนนต์แม่ที่ตั้งค่าเป็นหลักแหล่งอ้างอิง ซึ่งช่วยให้เราสามารถจัดทำส่วนทับซ้อนหรือปุ่มลอยต่าง ๆ ได้ตามต้องการ

การใช้งาน Custom Fonts ด้วย expo-font: การดาวน์โหลดและแสดงผลแบบอักษรแบรนด์เฉพาะตัวกระทำได้ผ่านชุดไลบรารี expo-font ร่วมกับ expo-splash-screen โดยเราจะต้องทำโครงสร้างการหน่วงเวลาเพื่อสั่งให้ระบบคงภาพ Splash Screen ค้างไว้ก่อนในขณะที่ตัวฟังก์ชัน

useFonts กำลังโหลดชุดอักษรพิเศษ เพื่อป้องกันการแสดงผลแบบอักษรผิดพลาดหรือเกิดการกระตุกเปลี่ยนแบบอักษรหลังจากเข้าหน้าแอปพลิเคชัน

การใช้งานระบบเวกเตอร์ไอคอนด้วย @expo/vector-icons: เพื่อประหยัดขนาดพื้นที่และความสามารถในการตอบสนองสเกลภาพ เวกเตอร์ไอคอนจากไลบรารี @expo/vector-icons (Expo, 2024) จึงเป็นทางเลือกหลักในการรวบรวมสัญลักษณ์สากลอย่าง Ionicons, Feather หรือ FontAwesome มาใช้งานในโปรเจกต์ ซึ่งสามารถนำรูปภาพที่ถูกต้องจากคู่มือค้นหารายชื่อไอคอน icons.expo.fyi มาใส่สีและกำหนดขนาดตามที่สไตล์ชีทกำหนดไว้ได้อย่างมีประสิทธิภาพ

การจัดการรูปภาพประสิทธิภาพสูงด้วย expo-image: คอมโพเนนต์ดั้งเดิมมีปัญหาด้านแคชข้อมูลภาพจากเน็ตเวิร์กและการกระตุกตอนปัดเลื่อนจอ ทำให้ Expo ได้พัฒนา expo-image ขึ้นมาทำงานทดแทนผ่านเทคโนโลยีฝั่งเนทีฟ (Glide และ SDWebImage) ซึ่งมอบฟีเจอร์การสร้างแคชจัดเก็บบนความจุไดรฟ์เครื่องอัตโนมัติ ทำให้ผู้พัฒนาสามารถจัดสรรการแสดงผลรูปภาพจำนวนมากได้อย่างรวดเร็วและประหยัดอินเทอร์เน็ตของผู้ใช้อย่างเด่นชัด

การใช้งาน Blurhash Placeholder และ Transition: การเสริมสร้างรอยต่อเวลาที่รูปภาพหลักกำลังอยู่ในกระบวนการดาวน์โหลดสามารถทำได้ผ่านสองกลยุทธ์หลัก คือการใช้รหัสข้อความจำลองเฉดสีขนาดเล็กอย่าง Blurhash แสดงผลขึ้นมาเป็นภาพเบลอตัวอย่างโดยไม่ต้องรอเน็ตเวิร์ก และระบบเล่นเอฟเฟกต์ Fade-in Transition ของ expo-image เพื่อทำการเปลี่ยนผ่านภาพจากแบบเบลอไปสู่ภาพจริงได้อย่างนุ่มนวลและลดอาการกระตุกทางสายตา

การรองรับโหมดมืดด้วย useColorScheme: โทรศัพท์มือถือยุคใหม่มักจะให้ผู้ใช้ปรับแต่งธีมสว่างหรือมืดตามใจชอบ ซึ่งเราสามารถดักฟังค่าระบบปฏิบัติการได้ผ่าน useColorScheme Hook ของ react-native เพื่อนำค่า light หรือ dark มาทำการสลับโทนสีของแอปพลิเคชันอย่างไดนามิกผ่านโครงสร้างออบเจกต์สไตล์ชีท ช่วยให้อินเทอร์เฟซมีความกลมกลืนตามความชอบของผู้ใช้งานเครื่องสมาร์ทโฟน

แบบฝึกหัดท้ายบทเรียน

แบบทดสอบนี้มีวัตถุประสงค์เพื่อประเมินความรู้และความเข้าใจเกี่ยวกับระบบการควบคุมสไตล์ขั้นสูง (Absolute Positioning), การโหลดและใช้งานแบบอักษรพิเศษ (Custom Fonts), การประยุกต์ใช้เวกเตอร์ไอคอน (@expo/vector-icons), การเพิ่มประสิทธิภาพจัดแสดงรูปภาพ (expo-image) และการรองรับโหมดมืด (Dark Mode / useColorScheme) ใน React Native Expo SDK 54

คำถามประจำบทเรียน (10 ข้อ)

ข้อ 1: ในการกำหนดสไตล์ของ React Native ข้อใดอธิบายพฤติกรรมของ `position: 'absolute'` ได้ถูกต้องที่สุด?

ก. ชิ้นงานจะขยับเยื้องจากจุดเดิมตามแถว Flexbox และยังคงจองพื้นที่ความกว้างและความสูงไว้

ข. ชิ้นงานจะหลุดออกจากโฟลว์การจัดแถว Flexbox ของหน้าจอ โดยสามารถระบุตำแหน่งทับซ้อนได้อย่างอิสระเทียบกับจุดเริ่มต้นของคอนเทนเนอร์แม่ตัวแรกสุดที่มีโครงสร้างล้อมรอบมัน

ค. ชิ้นงานจะถูกลบออกจากสเตตและการแสดงผลบนหน้าจอทันที

ง. ชิ้นงานจะขยายขนาดกว้าง 100% เสมอโดยอ้างอิงจากขนาดพิกเซลความสูงของหน้าจอระบบปฏิบัติการ

ข้อ 2: พร็อพส์บอกทิศทางใดต่อไปนี้ ไม่นิยม นำมาใช้ร่วมกับคอมโพเนนต์ที่กำหนดสไตล์เป็น `position: 'absolute'`?

ก. `top`

ข. `right`

ค. `flexDirection`

ง. `bottom`

ข้อ 3: การประยุกต์ใช้ไลบรารี `expo-font` ร่วมกับ `expo-splash-screen` เพื่อจุดประสงค์ใดเป็นสำคัญ?

ก. เพื่อบีบอัดขนาดไฟล์รูปภาพ PNG ในแอปพลิเคชันให้เล็กลง

ข. เพื่อให้ Splash Screen แสดงผลค้างไว้ในขณะที่แอปกำลังดาวน์โหลดฟอนต์เข้ามาเก็บในหน่วยความจำชั่วคราว ป้องกันปัญหากรอบอักษรพังหรือกระตุก (Flash of Unstyled Text)

ค. เพื่อเปลี่ยนสีหน้าจอหลักของโทรศัพท์ให้กลายเป็นสีธีมเข้มโดยทันที

ง. เพื่อสร้างไฟล์ติดตั้ง APK สำหรับส่งขึ้น Google Play Store

ข้อ 4: จากโค้ดเรียกใช้งาน `const [fontsLoaded, error] = useFonts({...})` หากฟอนต์ดาวน์โหลดพังหรือระบุพาธไฟล์ผิดพลาด ตัวแปรใดที่จะช่วยดักจับและส่งต่อสัญญาณสีแดงแจ้งปัญหาให้เราซ่อน Splash Screen เพื่อป้องกันหน้าจอค้างตลอดกาล?

ก. `fontsLoaded`

ข. `error`

ค. `useFonts`

ง. `SplashScreen`

ข้อ 5: แหล่งสืบค้นคำสั่งนำเข้า (Import) และรายชื่อไอคอนเวกเตอร์อย่างเป็นทางการของ Expo ที่สามารถตรวจสอบรูปทรง ขนาด และสีก่อนเขียนโค้ดได้คือเว็บไซต์ใด?

ก. google.com

ข. npmjs.com

ค. icons.expo.fyi

ง. expo.dev/fonts

ข้อ 6: เหตุใดเราจึงควรเลือกใช้งาน `@expo/vector-icons` ในการออกแบบเมนูหรือปุ่มสัญลักษณ์ แทนการใช้รูปภาพไฟล์ PNG ขนาดเล็ก?

ก. ไอคอนแบบเวกเตอร์สามารถย่อขยายขนาดและเปลี่ยนเฉดสีได้อย่างคมชัดโดยภาพไม่แตก และช่วยประหยัดเนื้อที่ขนาดไฟล์แอปพลิเคชันได้ดีกว่า

ข. ไอคอนเวกเตอร์ช่วยเชื่อมต่อบลูทูธของระบบมือถือเข้าหาอุปกรณ์เสริมได้ไวกว่า

ค. รูปภาพแบบ PNG สามารถปรับเปลี่ยนทิศทางการหมุนได้ดีกว่าเวกเตอร์

ง. เพื่อป้องกันไม่ให้ผู้ใช้งานสามารถก๊อปปี้โลโก้ของแอปได้

ข้อ 7: ข้อใดกล่าวถึงประโยชน์และความจำเป็นของการเปลี่ยนมาใช้คอมโพเนนต์ `Image` จากไลบรารี `expo-image` แทนคอมโพเนนต์ `Image` ดั้งเดิมของ React Native ได้อย่างถูกต้อง?

ก. เพื่อให้แอปพลิเคชันรันความละเอียดรูปภาพแบบ 8K ได้ลื่นไหลบนโทรศัพท์รุ่นเก่า

ข. เพื่อความสามารถในการแคชรูปภาพระดับเนทีฟ (Native Caching) ทั้งความจำสำรองและฮาร์ดดิสก์, ลดการใช้แบนด์วิดท์ และทำให้ปัดเลื่อนรายการที่มีภาพยาว ๆ ได้ลื่นไหลขึ้น

ค. เพื่อให้สามารถแต่งรูปภาพ ฟิลเตอร์สี และตัดภาพได้จากฝั่งโค้ดโปรเจกต์

ง. เพื่อปิดกั้นการใช้รูปจาก URL ภายนอก และอนุญาตเฉพาะภาพในเครื่องเท่านั้น

ข้อ 8: การทำงานของ "Blurhash" ในความสามารถของ `expo-image` มีหน้าที่หลักอย่างไรในการเพิ่มความพึงพอใจให้ผู้ใช้ (User Experience)?

ก. ช่วยสลับรูปภาพหลักให้หมุนเป็นวงกลมเร่งความเร็วการประมวลผล

ข. แปลงภาพจริงให้เป็นรหัสข้อความสั้น ๆ เพื่อนำมารันเป็นภาพตัวอย่างไล่เฉดสีเบลอ ๆ ทันทีในระหว่างที่รอดึงไฟล์ภาพใหญ่จากเน็ตเวิร์ก ช่วยแก้ปัญหาหน้าจอว่างเปล่า

ค. บังคับใส่เอฟเฟกต์เบลอบนหน้าจอเมื่อผู้ใช้พยายามแคปหน้าจอแอป

ง. ทำหน้าที่ย่อขนาดไฟล์ภาพจริงให้เหลือความกว้างเพียง 10 พิกเซลเก็บลงเซิร์ฟเวอร์

ข้อ 9: พร็อพส์ (Props) ตัวใดของ `expo-image` ที่ใช้ในการกำหนดเอฟเฟกต์เฟดภาพจริงขึ้นมาอย่างราบรื่นหลังจากดาวน์โหลดเสร็จสิ้น เพื่อไม่ให้แอปเกิดการกระตุกเปลี่ยนผ่านภาพกะทันหัน?

ก. `transition`

ข. `contentFit`

ค. `placeholder`

ง. `cachePolicy`

ข้อ 10: หากต้องการรับทราบการเปลี่ยนสีธีมระบบเครื่องของระบบปฏิบัติการมือถือปลายทาง (Light/Dark Mode) แบบเรียลไทม์ ผู้พัฒนาต้องเรียกใช้งาน React Native Hook ตัวใด?

ก. `useFonts`

ข. `useColorScheme`

ค. `useRoute`

ง. `useNavigation`

เฉลยคำตอบและคำอธิบายโดยละเอียด

เฉลย ข้อ 1: ตอบ ข.

• เหตุผลประกอบ:

• ก. ผิด: เป็นพฤติกรรมของ `position: 'relative'` ที่ยังคงครอบจองพื้นที่เดิมไว้ใน Layout flow

• ข. ถูก: `position: 'absolute'` จะดึงตัวมันเองออกจากโฟลว์ปกติ ทำให้สไตล์ตัวถัดไปขยับขึ้นมาแทนที่ และใช้พิกัด `top/bottom/left/right` ระบุการทับซ้อนเหนือนิ้วและชิ้นงานอื่น ๆ โดยยึดตามพิกัดของ Container แม่ที่มีการระบุขอบเขต

• ค. และ ง. ผิด: ไม่ได้เกี่ยวกับสเตตและไม่ได้บังคับขยาย 100% ตลอดเวลาแต่อย่างใด

เฉลย ข้อ 2: ตอบ ค.

• เหตุผลประกอบ:

• ก. ข. และ ง. ผิด: `top`, `bottom` และ `right` (รวมถึง `left`) เป็นหัวใจสำคัญของ Absolute Positioning เพื่อระบุพิกัดช่องไฟจากขอบแม่

• ค. ถูก: `flexDirection` เป็นพร็อพส์หลักของระบบจัดเรียง Flexbox Layout (เช่นจัดแนวแถวหรือแนวคอลัมน์) ซึ่งปกติคอมโพเนนต์ที่เป็น `absolute` จะหลุดออกจากระบบจัดเรียงนี้ จึงไม่นำมาประกาศคุมทิศทางตนเองโดยตรง (แต่มักใช้คุม Container แม่ หรือ Container ลูกภายใน absolute view อีกที)

เฉลย ข้อ 3: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** ไม่สามารถลดขนาดไฟล์ PNG ในโครงสร้างโฟลเดอร์ได้โดยตรง

• **ข. ถูก:** `expo-splash-screen` ช่วยบล็อกไม่ให้แอปปิดหน้าจอโลโก้ของระบบตั้งต้นก่อนที่สเตตัสการดาวน์โหลด Custom Fonts ของ `expo-font` จะเสร็จสิ้นลง ซึ่งหากโหลดเสร็จสมบูรณ์จึงปล่อยให้เปิดหน้าแอปจริง จะได้ฟอนต์ที่แสดงผลสวยงามทันทีไม่มีหน้ากระตุกอักษรเปลี่ยนรูป

• **ค. และ ง. ผิด:** ธีมเครื่องและฟังก์ชันการบิลด์ APK เป็นบทบาทหน้าที่ของตัวแปรและเอนจิ้นภายนอกตัวอื่น ๆ

เฉลย ข้อ 4: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** `fontsLoaded` จะเปลี่ยนเป็น `true` เมื่อฟอนต์ถูกโหลดผ่านเกณฑ์สำเร็จเท่านั้น

• **ข. ถูก:** ตัวแปร `error` จากอาร์เรย์ผลลัพธ์ของ `useFonts` จะเก็บสถานะข้อผิดพลาดเมื่อเกิดปัญหาโหลดไฟล์ฟอนต์ไม่ขึ้น ซึ่งเราต้องเขียนดักเช็ค `if (fontsLoaded || error) { SplashScreen.hideAsync(); }` เพื่อให้แอปข้ามไปเรนเดอร์หน้าจอปกติ ไม่ติดค้างอยู่ที่หน้า Splash Screen

• **ค. และ ง. ผิด:** `useFonts` เป็นเพียงฟังก์ชัน Hook และ `SplashScreen` เป็นอ็อบเจกต์คำสั่งปิดหน้ากาก ไม่ใช่ตัวแปรดักความผิดพลาด

เฉลย ข้อ 5: ตอบ ค.

• **เหตุผลประกอบ:**

• **ก. ข. และ ง. ผิด:** เป็นเว็บไซต์ค้นหาทั่วไป และไม่ใช่หน้าต่างเอกสารแสดงเวกเตอร์ไอคอนอย่างเจาะจง

• **ค. ถูก:** icons.expo.fyi คือสารบัญไอคอนอย่างเป็นทางการของ Expo ที่ช่วยให้เราค้นหาชื่อและคีย์เวิร์ดของชุดไอคอนต่าง ๆ จากคลังเนทีฟได้แม่นยำ

เฉลย ข้อ 6: ตอบ ก.

• **เหตุผลประกอบ:**

• **ก. ถูก:** เวกเตอร์ไอคอนถูกเก็บข้อมูลเป็นพิกัดทางคณิตศาสตร์ (Vector SVG/Font) ทำให้ย่อขยายขนาดได้ไม่จำกัดโดยไม่แตกพร่ามัว สามารถปรับแต่งสีสันได้เสรีผ่านโค้ดสไตล์ และมีขนาดข้อมูลที่เล็กกว่าไฟล์ภาพบิตแมป PNG อย่างมหาศาล

• **ข. ค. และ ง. ผิด:** ไม่เกี่ยวข้องกับสัญญาณการจับคู่บลูทูธ และการลากหมุนเวกเตอร์ทำได้ดีกว่า PNG รวมถึงไม่ใช่เครื่องมือป้องกันการก๊อปปี้สัญลักษณ์

เฉลย ข้อ 7: ตอบ ข.

• เหตุผลประกอบ:

• ก. ค. และ ง. ผิด: ไม่สามารถอัพสเกลคุณภาพหน้าจอเครื่อง หรือใช้ตัดแต่งแก้ไขภาพระดับโฟโต้ชอปได้ และรองรับรีโมท URL ปกติ

• ข. ถูก: `expo-image` พัฒนาขึ้นเพื่อแก้ปัญหาด้านประสิทธิภาพโดยใช้ Engine native ระดับสูง มีประสิทธิภาพสูงในการสร้าง Cache เก็บไฟล์ภาพลงฮาร์ดดิสก์เครื่องอัตโนมัติ ช่วยลดปัญหาความล่าช้าและการกินแรมตอนผู้ใช้เลื่อนดูหน้าต่างข้อมูลยาว ๆ

เฉลย ข้อ 8: ตอบ ข.

• เหตุผลประกอบ:

• ก. ค. และ ง. ผิด: ไม่เกี่ยวข้องกับกลไกความปลอดภัยการก๊อปปี้หน้าจอ หรือการลดความกว้างข้อมูลจริงบนเครื่องแม่ข่าย

• ข. ถูก: `Blurhash` คือรูปแบบการคำนวณที่แปลงภาพความละเอียดสูงให้เหลือสตริงอักษรขนาดสั้น เพื่อสร้างแถบสีพรีวิวเบลอ ๆ แสดงขึ้นมาแทนที่กล่องภาพเปล่าระหว่างโหลด ทำให้ผู้ใช้รู้สึกว่าแอปตอบสนองได้อย่างรวดเร็ว

เฉลย ข้อ 9: ตอบ ก.

• เหตุผลประกอบ:

• ก. ถูก: พร็อพส์ `transition` ของ `expo-image` รับค่าเวลาเป็นมิลลิวินาที (เช่น 600 = 0.6 วินาที) สำหรับกำหนดเวลาในการเล่นอนิเมชัน Fade-in เมื่อโหลดรูปจริงมาแสดงผลต่อจาก placeholder

• ข. ผิด: `contentFit` ใช้ปรับระดับสเกลภาพให้พอดีกล่อง (เช่น cover, contain)

• ค. ผิด: `placeholder` ใช้รับข้อมูลจำลองเพื่อรอโหลดภาพจริง

• ง. ผิด: `cachePolicy` ใช้จัดการนโยบายการจัดเก็บความจำสำรอง

เฉลย ข้อ 10: ตอบ ข.

• เหตุผลประกอบ:

• ก. ค. และ ง. ผิด: เป็นตัวจัดการฟอนต์, จัดระบบส้นทางเราเตอร์ และการนำทางตามลำดับ

• ข. ถูก: `useColorScheme` Hook ดึงค่าสภาพแวดล้อมสว่าง/มืด (`'light'` หรือ `'dark'`) ของระบบมือถือผู้ใช้งานกลับมาผูกใน Component เพื่อสลับสไตล์ชีทตามระบบการใช้งานในโหมดมืดได้ทันที

ปฏิบัติการท้ายบทเรียน: การสร้างการ์ดโปรไฟล์พรีเมียมรองรับระบบสลับธีม

ปฏิบัติการนี้จะฝึกทักษะการประยุกต์ใช้งานสไตล์ระดับสูงในระบบ React Native โดยผู้เรียนจะได้ออกแบบแอปพลิเคชันหน้า **แดชบอร์ดการ์ดโปรไฟล์ระดับพรีเมียม (Premium Profile Card Dashboard)** ที่โหลดฟอนต์พิเศษมาใช้งาน ตกแต่งจุดทับซ้อนไอคอนแบบสไตล์สัมบูรณ์ (`absolute`) โหลดรูปภาพโปรไฟล์ผ่านไลบรารี `expo-image` พร้อมรองรับการแสดงเฉดสีพรีวิวด้วยรหัส Blurhash และการรองรับโหมดมืด (Dark Mode) ตามเครื่องปลายทาง

1. วัตถุประสงค์การเรียนรู้

1. สามารถประยุกต์ใช้พิกัดตำแหน่งสัมบูรณ์ `position: 'absolute'` ในการวางเครื่องหมายออนไลน์ (Online Status Badge) บนรูปภาพโปรไฟล์
2. สามารถติดตั้ง โหลด และใช้งานแบบอักษรพิเศษ (Custom Fonts) ผ่านระบบของ `expo-font` ร่วมกับการใช้ `expo-splash-screen`
3. สามารถดึงและจัดสไตล์สัญลักษณ์เวกเตอร์ไอคอนด้วยไลบรารี `@expo/vector-icons`
4. สามารถเพิ่มประสิทธิภาพและความลื่นไหลการโหลดภาพโปรไฟล์ด้วย `expo-image` พร้อมกำหนดค่า `placeholder` (Blurhash) และการเฟดภาพด้วย `transition`
5. สามารถดักเซ็คและประยุกต์ใช้โทนสีกราฟิกตามธีมสว่างหรือธีมมืด (Dark Mode) ด้วย `useColorScheme` Hook

2. โจทย์และข้อกำหนดปฏิบัติการ (Requirements)

ผู้เรียนต้องสร้างโครงสร้างคอมโพเนนต์การ์ดโปรไฟล์ส่วนตัว (Profile Dashboard Screen) ที่มีเงื่อนไขดังต่อไปนี้:

1. **การโหลดทรัพยากรระบบ (Assets Loading):**
   - โหลดฟอนต์จำนวน 2 รูปแบบ ได้แก่ `Prompt-Regular` (สำหรับเนื้อหาทั่วไป) และ `Prompt-Bold` (สำหรับชื่อโปรไฟล์และหัวข้อสำคัญ)
   - แสดงหน้า Splash Screen ค้างไว้ และสั่งเปิดเผยหน้าแอปเมื่อโหลดฟอนต์เสร็จเท่านั้น หากเกิดข้อผิดพลาดในการโหลดไฟล์ฟอนต์ ต้องยกเลิกหน้า Splash เพื่อป้องกันหน้าจอค้างตลอดกาล
2. **การจัดอินเทอร์เฟซโปรไฟล์พรีเมียม (Premium Interface Layout):**
   - **กรอบภาพโปรไฟล์ (Avatar Container):** รันรูปวงกลมด้วย `expo-image` ดึงภาพจากลิงก์เน็ตเวิร์ก โดยตั้งค่า `placeholder` เป็นรหัส Blurhash และใช้ความเร็ว `transition` เท่ากับ 500 มิลลิวินาที

• **เครื่องหมายบอกสถานะการใช้งาน (Status Badge):** วางวงกลมสีเขียวนีออน (`#22c55e`) ขนาดเล็กระบุ "Online" ทับซ้อนลงบนมุมขวาล่างของรูปภาพวงกลมโปรไฟล์หลักด้วย `position: 'absolute'`

• **ข้อมูลส่วนตัว:** แสดงชื่อจริง ยศตำแหน่งงาน และประวัติสั้น ๆ โดยเรียกใช้ตระกูลอักษร `Prompt-Bold` และ `Prompt-Regular`

3. **ตารางชุดปุ่มคำสั่งด่วน (Action Grid):**

• แสดงชุดปุ่มแบบการ์ดขนาดเล็ก 3 ช่อง ได้แก่ "ส่งอีเมล", "ส่งข้อความ" และ "แชร์ข้อมูล"

• ในการ์ดย่อยแต่ละตัวให้เรียกสัญลักษณ์เวกเตอร์มาจาก `@expo/vector-icons` เช่น `mail`, `message-square`, `share-2`

4. **การสลับสไตล์ตามระบบเครื่อง (Theme Support):**

• ดึงสถานะธีมด้วย `useColorScheme()`

• หากระบบปฏิบัติการเป็นโหมดสว่าง (Light Theme) ให้ใช้สีพื้นหลังเป็น `#f8fafc` ตัวการ์ดสี `#ffffff` ตัวหนังสือสีเข้ม `#0f172a`

• หากระบบปฏิบัติการเป็นโหมดมืด (Dark Theme) ให้ใช้สีพื้นหลังเป็น `#0f172a` ตัวการ์ดสี `#1e293b` ตัวหนังสือสีขาว `#ffffff` และขอบสี `#334155`

3. ขั้นตอนการลงมือปฏิบัติ (Step-by-Step Tutorial)

ขั้นตอนที่ 1: ติดตั้งไลบรารีและจัดสรรทรัพยากรฟอนต์

เรียกใช้คำสั่งเพื่อติดตั้งไลบรารีหลักทั้งหมดในโฟลเดอร์โปรเจกต์:

```
npx expo install expo-font expo-splash-screen expo-image
```

หมายเหตุ: ตรวจสอบให้แน่ใจว่าได้จัดเตรียมไฟล์ฟอนต์ `Prompt-Regular.ttf` และ `Prompt-Bold.ttf` บันทึกเก็บไว้ที่ไดเรกทอรี `assets/fonts/` ของแอปพลิเคชัน

ขั้นตอนที่ 2: โครงสร้างนำเข้าโมดูลและเริ่มหน่วงเวลา Splash Screen

เขียนเริ่มการโหลดฟอนต์ในหน้าแรกสุด:

```
import { useEffect } from 'react';
import { View, Text, StyleSheet, useColorScheme, Pressable } from 'react-native';
import { useFonts } from 'expo-font';
import * as SplashScreen from 'expo-splash-screen';
import { Image } from 'expo-image';
import { Feather } from '@expo/vector-icons';

// ตรึงหน้ารูปภาพโลโก้ไว้ก่อนรอข้อมูลแบบอักษรเสร็จสิ้น
```

```
SplashScreen.preventAutoHideAsync();
```

ขั้นตอนที่ 3: ออกแบบสเตตัส Badge ลอยสัมบูรณ์ทับบน Avatar

กำหนดรูปสไตล์รูปภาพให้มีกรอบความกว้างและสูงเป็นวงกลม และใช้ absolute ปักหมุดลูกด้านล่าง:

```
// กล่องครอบรูปวงกลมมีพิกัดเป็น relative
<View style={styles.avatarWrapper}>
  <Image
    source="https://images.unsplash.com/photo-1534528741775-53994a69daeb?w=400"
    placeholder={{ blurhash: 'LKN]~^%hS$of9G-pB%-;U_RPn%s:' }}
    style={styles.avatar}
    transition={500}
 />
  {/* จุดวงกลมออนไลน์วางปักหมุดไว้ที่มุมขวาล่าง */}
  <View style={styles.onlineBadge} />
</View>
```

4. โค้ดเฉลยฉบับสมบูรณ์ (Complete Solution)

ผู้เรียนสามารถนำโค้ดเฉลยเต็มระบบชุดนี้ไปบันทึกทับในไฟล์หลักของแอปพลิเคชันเพื่อใช้ทดสอบ โดยสลับการตั้งค่าโหมดหน้าจอมือถือจำลอง (Simulator) ไปมาระหว่าง Light และ Dark Mode เพื่อตรวจผลลัพธ์:

```
import { useEffect } from 'react';
import { StyleSheet, Text, View, useColorScheme, Pressable, ScrollView,
SafeAreaView } from 'react-native';
import { useFonts } from 'expo-font';
import * as SplashScreen from 'expo-splash-screen';
import { Image } from 'expo-image';
import { Feather, Ionicons } from '@expo/vector-icons';

// 1. ตรึงหน้า Splash Screen ไม่ให้หายไปกะทันหันก่อนดาวน์โหลดฟอนต์เสร็จ
SplashScreen.preventAutoHideAsync();

export default function App() {
 // 2. เรียกใช้งานระบบโหลดฟอนต์ไทยยอดนิยม (Prompt)
  const [fontsLoaded, error] = useFonts({
    'Prompt-Regular': require('../../assets/fonts/Prompt-Regular.ttf'),
    'Prompt-Bold': require('../../assets/fonts/Prompt-Bold.ttf'),
  });

  useEffect(() => {
  // 3. ทันทีที่โหลดฟอนต์พร้อม (หรือตรวจพบ Error) ให้ทำการซ่อน Splash Screen
    if (fontsLoaded || error) {
```

```
      SplashScreen.hideAsync();
    }
  }, [fontsLoaded, error]);

 // 4. หากฟอนต์ยังเรนเดอร์ไม่เสร็จสิ้น ให้รอชั่วขณะ
  if (!fontsLoaded && !error) {
    return null;
  }

 // 5. ดักจับเฉดสีธีมเครื่องปลายทางด้วย Hook
  const colorScheme = useColorScheme();
  const isDark = colorScheme === 'dark';

 // 6. กำหนดชุดคู่สีให้ปรับเปลี่ยนตามสเตตัสเครื่อง
  const theme = {
    background: isDark ? '#0f172a' : '#f8fafc',
    cardBackground: isDark ? '#1e293b' : '#ffffff',
    textPrimary: isDark ? '#ffffff' : '#0f172a',
    textSecondary: isDark ? '#94a3b8' : '#64748b',
    borderColor: isDark ? '#334155' : '#e2e8f0',
    primaryColor: '#38bdf8',
  };

  return (
    <SafeAreaView style={[styles.safeArea, { backgroundColor:
theme.background }]}>
      <ScrollView contentContainerStyle={styles.scrollContainer}>
        {/* หัวข้อด้านบนสุด */}
        <View style={styles.header}>
          <Text style={[styles.headerTitle, { color: theme.textPrimary
}]}>แดชบอร์ดส่วนตัว</Text>
          <Text style={[styles.headerSubtitle, { color:
theme.textSecondary }]}>Premium Profile Card</Text>
        </View>

        {/* การ์ดโปรไฟล์หลัก */}
        <View style={[styles.profileCard, { backgroundColor:
theme.cardBackground, borderColor: theme.borderColor }]}>

          {/* ส่วนกรอบตกแต่งรูปวงกลมด้วยสไตล์สัมบูรณ์ */}
          <View style={styles.avatarContainer}>
            <View style={styles.avatarWrapper}>
              <Image
           source="https://images.unsplash.com/photo-1494790108377-
be9c29b29330?w=400"
                placeholder={{ blurhash: 'LKN]~^%hS$of9G-pB%-;U_RPn%s:' }}
                style={styles.avatar}
                contentFit="cover"
```

```
                transition={500}
        />
              {/* สไตล์ Absolute วางทับมุมโปรไฟล์ด้านล่างขวา */}
              <View style={[styles.statusBadge, { borderColor:
theme.cardBackground }]} />
            </View>
          </View>

          {/* รายละเอียดข้อความจำลองโปรไฟล์ */}
          <Text style={[styles.profileName, { color: theme.textPrimary
}]}>ชนิดาภา รัตนพิมล</Text>
          <Text style={[styles.profileRole, { color: theme.primaryColor
}]}>Lead UX/UI Designer</Text>
          <Text style={[styles.profileBio, { color: theme.textSecondary
}]}>
      ชื่นชอบการออกแบบโมบายแอปพลิเคชันระดับพรีเมียม สนุกกับการพัฒนาอินเทอร์เฟซด้วยสไตล์อนิเมชันลื่นไหลและดีไซน์
โหมดมืดแสนดึงดูดสายตา
          </Text>

          {/* แถบเส้นขีดคั่นกลางของการ์ด */}
          <View style={[styles.divider, { backgroundColor:
theme.borderColor }]} />

          {/* สถิติตัวเลขย่อ */}
          <View style={styles.statsRow}>
            <View style={styles.statCol}>
              <Text style={[styles.statNumber, { color: theme.textPrimary
}]}>48</Text>
              <Text style={[styles.statLabel, { color: theme.textSecondary
}]}>โครงการ</Text>
            </View>
            <View style={styles.statCol}>
              <Text style={[styles.statNumber, { color: theme.textPrimary
}]}>12.4K</Text>
              <Text style={[styles.statLabel, { color: theme.textSecondary
}]}>ผู้ติดตาม</Text>
            </View>
            <View style={styles.statCol}>
              <Text style={[styles.statNumber, { color: theme.textPrimary
}]}>150</Text>
              <Text style={[styles.statLabel, { color: theme.textSecondary
}]}>รีวิวดีเยี่ยม</Text>
            </View>
          </View>
        </View>

        {/* เมนูการ์ดคำสั่งปุ่มสัมผัส (Action Grid) */}
        <View style={styles.actionGrid}>
          <Pressable style={[styles.actionCard, { backgroundColor:
```

```
theme.cardBackground, borderColor: theme.borderColor }]}>
            <View style={[styles.iconCircle, { backgroundColor: '#e0f2fe'
}]}>
              <Feather name="mail" size={20} color="#0284c7" />
            </View>
            <Text style={[styles.actionLabel, { color: theme.textPrimary
}]}>ส่งอีเมล</Text>
          </Pressable>

          <Pressable style={[styles.actionCard, { backgroundColor:
theme.cardBackground, borderColor: theme.borderColor }]}>
            <View style={[styles.iconCircle, { backgroundColor: '#f0fdf4'
}]}>
              <Ionicons name="chatbubble-ellipses-outline" size={20}
color="#16a34a" />
            </View>
            <Text style={[styles.actionLabel, { color: theme.textPrimary
}]}>ส่งข้อความ</Text>
          </Pressable>

          <Pressable style={[styles.actionCard, { backgroundColor:
theme.cardBackground, borderColor: theme.borderColor }]}>
            <View style={[styles.iconCircle, { backgroundColor: '#fdf2f8'
}]}>
              <Feather name="share-2" size={20} color="#db2777" />
            </View>
            <Text style={[styles.actionLabel, { color: theme.textPrimary
}]}>แชร์ข้อมูล</Text>
          </Pressable>
        </View>
      </ScrollView>
    </SafeAreaView>
  );
}

const styles = StyleSheet.create({
  safeArea: {
    flex: 1,
  },
  scrollContainer: {
    padding: 24,
  },
  header: {
    marginBottom: 24,
    alignItems: 'center',
  },
  headerTitle: {
    fontSize: 24,
    fontFamily: 'Prompt-Bold',
  },
```

```
  headerSubtitle: {
    fontSize: 14,
    fontFamily: 'Prompt-Regular',
    marginTop: 4,
  },
  profileCard: {
    borderRadius: 24,
    padding: 24,
    alignItems: 'center',
    borderWidth: 1,
    shadowColor: '#000',
    shadowOffset: { width: 0, height: 4 },
    shadowOpacity: 0.1,
    shadowRadius: 10,
    elevation: 5,
  },
  avatarContainer: {
    marginBottom: 16,
  },
  avatarWrapper: {
    position: 'relative', // วางเพื่อรองรับพิกัด Absolute ของ Badge ลูก
  },
  avatar: {
    width: 100,
    height: 100,
    borderRadius: 50,
    backgroundColor: '#cbd5e1',
  },
  statusBadge: {
    position: 'absolute', // หลุดออกจากโฟลว์เพื่อทับซ้อนมุมโปรไฟล์
    bottom: 2,
    right: 2,
    width: 22,
    height: 22,
    borderRadius: 11,
    backgroundColor: '#22c55e', // สีเขียวนีออนบอกสถานะ Online
    borderWidth: 3,             // เติมขอบตัดเพื่อป้องกันการกลืนสี
  },
  profileName: {
    fontSize: 20,
    fontFamily: 'Prompt-Bold',
    marginBottom: 4,
  },
  profileRole: {
  fontSize: 14,
    fontFamily: 'Prompt-Bold',
```

```
    marginBottom: 16,
  },
  profileBio: {
    fontSize: 14,
    fontFamily: 'Prompt-Regular',
    textAlign: 'center',
    lineHeight: 22,
    marginBottom: 20,
  },
  divider: {
    height: 1,
    width: '100%',
    marginBottom: 20,
  },
  statsRow: {
    flexDirection: 'row',
    justifyContent: 'space-around',
    width: '100%',
  },
  statCol: {
    alignItems: 'center',
  },
  statNumber: {
    fontSize: 18,
    fontFamily: 'Prompt-Bold',
  },
  statLabel: {
    fontSize: 12,
    fontFamily: 'Prompt-Regular',
    marginTop: 2,
  },
  actionGrid: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    marginTop: 20,
  },
  actionCard: {
    flex: 1,
    marginHorizontal: 4,
    borderRadius: 16,
    paddingVertical: 16,
    alignItems: 'center',
    borderWidth: 1,
  },
  iconCircle: {
    width: 44,
    height: 44,
    borderRadius: 22,
    justifyContent: 'center',
```

```
    alignItems: 'center',
    marginBottom: 8,
  },
  actionLabel: {
    fontSize: 13,
    fontFamily: 'Prompt-Regular',
  },
});
```

5. การวิเคราะห์สรุปผลการประเมินปฏิบัติการ

เมื่อรันโครงการสำเร็จ ให้ผู้เรียนประเมินตรวจสอบเกณฑ์การตรวจวัดดังนี้:

1. ฟอนต์ของตัวอักษรทั้งหมดในแอปพลิเคชันต้องใช้แบบอักษร `Prompt` และ Splash Screen ต้องซ่อนอย่างสมบูรณ์โดยที่ไม่มีการแจ้งพังหรือค้างหน้าจอขาว

2. รูปภาพแสดงผลโปรไฟล์คมชัด ลื่นไหลไม่กะพริบตา และเล่นเอฟเฟกต์เฟดตอนเริ่มหน้าจอ

3. เมื่อเข้าที่ส่วนตั้งค่าเปลี่ยนธีมของเครื่องให้เป็น Dark Mode องค์ประกอบหน้าจอทั้งหมดต้องสับสีดำสลัวโดยอัตโนมัติ

เอกสารอ้างอิง

Expo. (2024). *Expo Image*. จาก https://docs.expo.dev/versions/v54.0.0/sdk/image/

Expo. (2024). *Icons*. จาก https://docs.expo.dev/versions/v54.0.0/guides/icons/

Expo. (2024). *Use custom fonts*. จาก https://docs.expo.dev/versions/v54.0.0/develop/user-interface/fonts/

React Native. (2024). *Style*. จาก https://reactnative.dev/docs/style